

Compiling a Parametric C Raytracer to AArch64

Staged semantic specialization and a fixed-point sphere kernel

16 August 2026

Abstract

This note expands the semantic-description compiler into a small raytracer compiler instance. A high-level description specifies image dimensions, sphere geometry, and target ABI policy. The compiler specializes that description into AArch64 containing nested render loops, a per-pixel sphere-intersection kernel, an explicit output hook, and a Linux syscall helper. Quoting and unquoting the description produces byte-identical output to direct compilation, demonstrating the self-applicable staging property.

1 Raytracer semantic description

The source-level description is

$$D_{ray} = \{W, H, c_x, c_y, r, \text{entry}, \text{syscallABI}\}.$$

The sample uses $W = 64$, $H = 48$, center $(32, 24)$, and radius 20. The ray model is orthographic and fixed point: a pixel (x, y) hits when

$$(x - c_x)^2 + (y - c_y)^2 < r^2.$$

The kernel returns intensity 1 for a hit and 0 otherwise.

2 Generated target structure

The compiler emits a function with the following ABI:

$$\text{raytrace_pixel}(x_0, x_1) \rightarrow x_0,$$

where $x_0 = x$, $x_1 = y$, followed by

$$\text{pixel_sink}(x_0, x_1, x_2),$$

where the arguments are intensity, x, and y. The sink is intentionally explicit: a C variant may implement it using PPM serialization, a framebuffer, or ‘write(2)’ without changing the ray kernel.

The generated control flow is a conventional nested loop:

```
.Lrow:
    cmp x19, #48
    b.ge .Lfinish
.Lcolumn:
    cmp x20, #64
    b.ge .Lnextrow
    bl raytrace_pixel
    bl pixel_sink
    b .Lcolumn
```

The pixel kernel is:

```
sub x2, x0, #32
mul x2, x2, x2
sub x3, x1, #24
mul x3, x3, x3
add x2, x2, x3
mov x3, #400
cmp x2, x3
cset x0, lt
ret
```

3 Staged compiler generation

The description is typed code:

$$\text{quote} : D_{ray} \rightarrow \text{Code}(D_{ray}), \quad \text{unquote} : \text{Code}(D_{ray}) \rightarrow D_{ray}.$$

The staged compiler is

$$\text{compileStaged}(\hat{D}) = \text{compile}(\text{unquote}(\hat{D})).$$

The executable compares direct and staged output as strings. Equality establishes that the semantic description is sufficient to reconstruct the same target compiler instance at the next stage.

4 Verification

The implementation is `raytracer_arm64.cpp`. It was compiled with:

```
g++ -std=c++23 -Wall -Wextra -pedantic -O2 raytracer_arm64.cpp -o raytracer_arm64
./raytracer_arm64 > raytracer_arm64.s
```

The executable reports:

```
raytracer staged equivalence: PASS
```

Structural checks confirm the presence of ‘raytrace’, the row and column loops, ‘raytrace_pixel’, ‘pixel_sink’, the radius-square constant 400, and ‘sys_write’.

5 Limitations

This is a compiler kernel, not a complete image-producing runtime. It does not yet lower floating point, triangles, recursion, acceleration structures, or a concrete PPM sink. The fixed-point sphere kernel is intentionally small and auditable. The extension path is to add typed vector and image-buffer descriptions, explicit memory effects, and a verified lowering for the output sink while preserving the staged description boundary.